

Spor Salonu Yönetim Sistemi

Ders Kitabı Tarzı Tutorial

Bu doküman Flask, PostgreSQL, SQL, Jinja, HTML/CSS ve Chart.js kullanılarak geliştirilen projenin öğretici açıklamasıdır.

Önsöz

Bu PDF, `tutorial.md` dosyasından ders kitabı görünümünde hazırlanmıştır. Amaç, projeyi yalnızca çalıştırma adımlarıyla değil; kullanılan teknolojiler, dosya yapısı, SQL syntax'ı, Flask route mantığı ve arayüz katmanıyla birlikte öğretici şekilde açıklamaktır.

İçindekiler

1 Projenin Genel Mantığı	2
2 Kullanılan Teknolojiler	3
2.1 Python	3
2.2 Flask	3
2.3 PostgreSQL	4
2.4 SQL	4
2.5 psycopg	5
2.6 HTML	5
2.7 Jinja	5
2.8 CSS	5
2.9 JavaScript ve Chart.js	6
3 Klasör Yapısı	7
4 Kurulum Mantığı	9
4.1 Sanal Ortam Nedir?	9
4.2 requirements.txt	9
4.3 .env Dosyası	10
5 PostgreSQL Kurulumu ve Veritabanı Hazırlığı	11
6 schema.sql Dosyası	13
6.1 DROP TABLE	13
6.2 CREATE TABLE	13
6.3 Primary Key	14
6.4 Foreign Key	14
6.5 CHECK Constraint	15
6.6 UNIQUE Constraint	15
6.7 Index	15
7 Trigger ve Function Mantığı	16
7.1 Function Syntax	16
7.2 Üyelik Tutarı Trigger'ı	17
7.3 Atama Kontrol Trigger'ı	17
7.4 DEFERRABLE Trigger	17

8 seed.sql Dosyası	19
9 db.py Dosyası	20
9.1 load_dotenv	20
9.2 get_database_url	20
9.3 get_conn	20
9.4 transaction	21
9.5 fetch_all ve fetch_one	21
10 app.py Dosyası	23
10.1 Import Bölümü	23
10.2 Flask App Oluşturma	24
10.3 Decorator Nedir?	24
10.4 login_required Decorator'ı	24
10.5 Template Filter	25
10.6 Login Route'u	25
10.7 Dashboard Route'u	26
10.8 JSON API Route'ları	26
10.9 CRUD Nedir?	26
11 Route'ların Görevleri	28
12 Üyelik Oluşturma Akışı	29
13 Ödeme Akışı	30
14 Yönetici Loglama	31
15 templates Klasörü	32
15.1 base.html	32
15.2 extends Kullanımı	32
15.3 Jinja Değişkenleri	33
15.4 Jinja Döngüleri	33
15.5 Jinja Koşulları	33
16 static/css/app.css Dosyası	34
16.1 CSS Değişkenleri	34
16.2 Layout	34
16.3 Grid Kullanımı	35
16.4 Responsive Tasarım	35
17 static/js/dashboard.js Dosyası	36
17.1 fetch	36
17.2 Promise.all	36
17.3 Chart.js	36

18 SQL Sorgu Mantığı	38
18.1 SELECT	38
18.2 WHERE	38
18.3 JOIN	38
18.4 LEFT JOIN	38
18.5 GROUP BY	39
18.6 Aggregate Fonksiyonlar	39
18.7 COALESCE	39
19 Dashboard Metrikleri	40
20 Sorgular Ekranı	41
21 İş Kuralları	42
21.1 Tek Aktif Üyelik	42
21.2 Profesyonel Plan	42
21.3 VIP Plan	42
21.4 Ücret Hesaplama	42
22 Güvenlik ve Doğru Uygulama Notları	44
22.1 Şifre Hashleme	44
22.2 SQL Parametreleri	44
22.3 Session	44
23 Uygulamayı Çalıştırma	45
24 Sunum İçin Anlatım Akışı	46
25 Dosya Dosya Kısa Özet	47
26 Bu Projeden Öğrenilen Ana Konular	48
27 HTTP, Request ve Response Mantığı	49
28 Route, Template ve SQL Eşleştirme Tablosu	51
29 Werkzeug ve Şifre Hashleme	53
30 Flash Mesajları ve Hata Gösterimi	54
31 seed.sql İçindeki Transaction ve setval Mantığı	55
32 psql Komut Satırı Syntax'ı	56
33 Neden Raw SQL Kullanıldı?	57
34 Projede Bilerek Basit Tutulan Noktalar	58
35 Sonuç	59

Bu dosya, projeyi hiç bilmeyen birine konu anlatır gibi açıklamak için hazırlanmıştır. Amaç yalnızca uygulamayı çalıştırmak değil; projede kullanılan teknolojileri, dosyaları, SQL yapısını, Flask mantığını, HTML/CSS/Jinja kullanımını ve dashboard grafiklerinin nasıl üretildiğini adım adım anlamaktır.

Proje konusu bir spor salonunun üyelik, çalışan, ödeme, atama ve yönetici log süreçlerini yönetmektir. Uygulama web arayüzü üzerinden çalışır, verileri PostgreSQL veritabanında saklar ve dashboard ekranında Chart.js ile grafik gösterir.

Bölüm 1

Projenin Genel Mantığı

Bu proje üç ana parçadan oluşur:

1. Backend: Python ve Flask ile yazılmıştır.
2. Veritabanı: PostgreSQL kullanır.
3. Frontend: HTML, CSS, Jinja template ve küçük JavaScript parçalarından oluşur.

Kullanıcı tarayıcıdan uygulamaya girer. Flask gelen isteği alır, gerekiyorsa PostgreSQL'e SQL sorgusu gönderir, veriyi alır ve HTML template içine yerleştirerek cevap döner.

Basit akış şu şekildedir:

```
11  
Tarayc -> Flask route -> PostgreSQL sorgusu -> Template render -> 11Tarayc
```

Örneğin kullanıcı dashboard ekranını açtığında:

1. Tarayıcı /dashboard adresine istek gönderir.
2. Flask içindeki dashboard() fonksiyonu çalışır.
3. PostgreSQL'den aktif sporcu, aylık gelir ve yaklaşan üyelik bitişleri çekilir.
4. dashboard.html template'i bu verilerle doldurulur.
5. Kullanıcının tarayıcısına HTML sayfası gönderilir.

Bölüm 2

Kullanılan Teknolojiler

2.1 Python

Python, bu projede backend tarafını yazmak için kullanılmıştır. Backend, kullanıcının yaptığı işlemleri karşılayan ve veritabanıyla konuşan kısımdır.

Python dosyaları:

```
app.py  
db.py
```

Python bu projede şunları yapar:

- Flask uygulamasını başlatır.
- Kullanıcı girişini kontrol eder.
- HTML sayfalarını render eder.
- PostgreSQL'e sorgu gönderir.
- Formlardan gelen verileri işler.
- Yönetici loglarını kaydeder.

2.2 Flask

Flask, Python ile web uygulaması geliştirmeyi sağlayan hafif bir framework'tür.

Flask şu soruya cevap verir:

```
11  
Kullanc şu URL'ye giderse hangi Python fonksiyonu ıřçalacak?
```

Örneğın:

```
@app.get("/dashboard")  
def dashboard():  
    return render_template("dashboard.html")
```

Bu kod şu anlama gelir:

- Kullanıcı /dashboard adresine GET isteğı atarsa

- `dashboard()` fonksiyonu çalışır
- Sonuç olarak `dashboard.html` sayfası döndürülür

2.3 PostgreSQL

PostgreSQL, ilişkisel veritabanı yönetim sistemidir. Projedeki kalıcı veriler burada tutulur.

Örneğin:

- Sporcular
- Çalışanlar
- Üyelikler
- Ödemeler
- Yönetici kayıtları
- Yönetici işlem logları

PostgreSQL sadece veri saklamaz. Bu projede bazı iş kuralları da PostgreSQL seviyesinde korunur:

- Aynı sporcuya aynı anda iki aktif üyelik açılamaz.
- Profesyonel planda diyetisyen zorunludur.
- VIP planda diyetisyen ve PT zorunludur.
- Üyelik tutarı plan ve süre katsayısından hesaplanır.

2.4 SQL

SQL, veritabanıyla konuşmak için kullanılan dildir.

Örneğin veri çekmek için:

```
SELECT * FROM sporcu;
```

Veri eklemek için:

```
INSERT INTO sporcu (ad, soyad, telefon)
VALUES ('Ali', 'Yildiz', '05320000001');
```

Bu projede SQL iki ana dosyada kullanılır:

```
schema.sql
seed.sql
```

Flask uygulaması içinde de birçok raw SQL sorgusu bulunur.

2.5 psycopg

psycopg, Python ile PostgreSQL arasında bağlantı kuran kütüphanedir.

Basit fikir şudur:

```
Python kodu SQL sorgusu yazar -> psycopg bunu PostgreSQL'e gönderir -> sonucu
Python'a döndürür
```

Projede bu bağlantı mantığı db.py dosyasında toplanmıştır.

2.6 HTML

HTML, sayfanın iskeletini oluşturur.

Örneğin bir form:

```
<form method="post">
  <input name="email">
  <button type="submit">ŞGiri Yap</button>
</form>
```

Bu projede HTML dosyaları templates/ klasöründedir.

2.7 Jinja

Jinja, Flask'ın kullandığı template motorudur. HTML içinde Python'dan gelen verileri göstermeyi sağlar.

Örneğin:

```
<h1>{{ aktif_yonetici }}</h1>
```

Burada aktif_yonetici Python tarafından template'e gönderilen bir değişkendir.

Jinja ile döngü de yazılabilir:

```
{% for sporcu in sporcular %}
  <tr>
    <td>{{ sporcu.ad }}</td>
    <td>{{ sporcu.soyad }}</td>
  </tr>
{% endfor %}
```

2.8 CSS

CSS, sayfaların görsel tasarımını oluşturur.

Bu projede tek ana CSS dosyası vardır:

```
static/css/app.css
```

CSS ile:

- Sidebar tasarımı
- Dashboard kartları
- Tablolar
- Formlar
- Butonlar
- Mobil uyumluluk

tasarlanmıştır.

2.9 JavaScript ve Chart.js

JavaScript, tarayıcı tarafında çalışan dildir. Bu projede JavaScript özellikle dashboard grafiklerini oluşturmak için kullanılmıştır.

Chart.js ise grafik çizmek için kullanılan bir JavaScript kütüphanesidir.

Bu projede:

```
static/js/dashboard.js
```

dosyası Flask API endpoint'lerinden JSON veri çeker ve grafik oluşturur.

Bölüm 3

Klasör Yapısı

Proje klasörü şu şekildedir:

```
spor_salonu_app/  
  app.py  
  db.py  
  schema.sql  
  seed.sql  
  requirements.txt  
  README.md  
  sql_islemleri_raporu.md  
  tutorial.md  
  .env  
  .env.example  
  templates/  
    base.html  
    login.html  
    dashboard.html  
    sporcular.html  
    sporcu_form.html  
    calisanlar.html  
    calisan_form.html  
    uyelik_form.html  
    uyelik_detay.html  
    odemeler.html  
    odeme_form.html  
    sorgular.html  
    loglar.html  
  static/  
    css/  
      app.css  
    js/  
      dashboard.js
```

Bu yapı Flask projeleri için klasik bir ayrımıdır:

- Python dosyaları backend mantığını taşır.
- templates/ HTML/Jinja dosyalarını taşır.

- `static/` CSS ve JavaScript gibi statik dosyaları taşır.
- SQL dosyaları veritabanı kurulumunu taşır.

Bölüm 4

Kurulum Mantığı

Projeyi çalıştırmak için iki temel sistem gerekir:

1. Python sanal ortamı
2. PostgreSQL veritabanı

4.1 Sanal Ortam Nedir?

Python sanal ortamı, projenin ihtiyaç duyduğu paketleri ayrı bir klasörde tutar.

Bu proje için sanal ortam:

```
.venv/
```

klasörüdür.

Sanal ortamı aktif etmek için:

```
source .venv/bin/activate
```

Terminal başında (.venv) görürsen sanal ortam aktiftir.

4.2 requirements.txt

requirements.txt, projenin ihtiyaç duyduğu Python paketlerini listeler.

Bu projede:

```
flask==3.1.3  
psycpg[binary]==3.3.4  
python-dotenv==1.2.2
```

Bu paketleri yüklemek için:

```
pip install -r requirements.txt
```

Paketlerin görevleri:

Paket	Görev
flask	Web uygulamasını çalıştırır.

psycopg[binary]	Python ile PostgreSQL bağlantısı kurar.
python-dotenv	.env dosyasındaki ayarları okur.

4.3 .env Dosyası

.env, gizli veya ortama özel ayarları tutar.

Projede örnek:

```
SECRET_KEY=dev-secret-change-me
DATABASE_URL=postgresql://spor_app:spor_app_123@localhost:5432/spor_salonu_db
FLASK_ENV=development
```

Burada:

- SECRET_KEY: Flask session güvenliği için kullanılır.
- DATABASE_URL: PostgreSQL bağlantı bilgisidir.
- FLASK_ENV: geliştirme ortamını belirtir.

DATABASE_URL parçalanırsa:

```
postgresql://kullanici:sifre@host:port/veritabani
```

Bu projede:

Parça	Değer
Kullanıcı	spor_app
Şifre	spor_app_123
Host	localhost
Port	5432
Veritabanı	spor_salonu_db

Bölüm 5

PostgreSQL Kurulumu ve Veritabanı Hazırlığı

PostgreSQL kurmak için:

```
sudo apt install postgresql postgresql-contrib
```

PostgreSQL yönetici terminaline girmek için:

```
sudo -u postgres psql
```

Sonra veritabanı kullanıcısı ve veritabanı oluşturulur:

```
CREATE USER spor_app WITH PASSWORD 'spor_app_123';
CREATE DATABASE spor_salonu_db OWNER spor_app;
GRANT ALL PRIVILEGES ON DATABASE spor_salonu_db TO spor_app;
\c spor_salonu_db
GRANT ALL ON SCHEMA public TO spor_app;
\q
```

Bu komutların anlamı:

Komut	Anlamı
CREATE USER	PostgreSQL içinde yeni kullanıcı oluşturur.
CREATE DATABASE	Yeni veritabanı oluşturur.
OWNER spor_app	Veritabanının sahibini uygulama kullanıcısı yapar.
GRANT	Yetki verir.
\c	Başka veritabanına bağlanır.
\q	psql'den çıkar.

Sonra şema ve demo veri yüklenir:

```
psql "postgresql://spor_app:spor_app_123@localhost:5432/spor_salonu_db" -f
  schema.sql
psql "postgresql://spor_app:spor_app_123@localhost:5432/spor_salonu_db" -f
  seed.sql
```


Burada -f, dosyadaki SQL komutlarını çalıştır anlamına gelir.

Bölüm 6

schema.sql Dosyası

schema.sql, veritabanının yapısını kurar.

Bu dosyada:

- Tablolar oluşturulur.
- Primary key tanımlanır.
- Foreign key ilişkileri kurulur.
- Check constraint'ler eklenir.
- Index'ler oluşturulur.
- Trigger function'ları yazılır.
- Trigger'lar bağlanır.

6.1 DROP TABLE

Dosyanın başında şu tip komutlar vardır:

```
DROP TABLE IF EXISTSyonetici_log CASCADE;  
DROP TABLE IF EXISTSodeme CASCADE;
```

Bu komutlar eski tabloları siler.

IF EXISTS şu anlama gelir:

Tablo varsa sil, yoksa hata verme.

CASCADE şu anlama gelir:

Bu tabloya gıbal şilikileri de dikkate alarak sil.

Bu yapı, projeyi tekrar tekrar kurarken kolaylık sağlar.

6.2 CREATE TABLE

Tablo oluşturmak için CREATE TABLE kullanılır.

Örnek:

```
CREATE TABLE sporcu (
    sporcu_id BIGSERIAL PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    soyad VARCHAR(50) NOT NULL,
    telefon VARCHAR(20) NOT NULL UNIQUE,
    kayit_tarihi DATE NOT NULL DEFAULT CURRENT_DATE,
    aktif_mi BOOLEAN NOT NULL DEFAULT TRUE
);
```

Burada:

Syntax	Anlamı
BIGSERIAL	Otomatik artan büyük sayı üretir.
PRIMARY KEY	Her satırı benzersiz tanımlar.
VARCHAR(50)	En fazla 50 karakterlik metin tutar.
NOT NULL	Boş bırakılamaz.
UNIQUE	Aynı değer iki kez girilemez.
DATE	Tarih tutar.
DEFAULT	Değer verilmezse otomatik değer kullanır.
BOOLEAN	TRUE veya FALSE tutar.

6.3 Primary Key

Primary key, tablodaki her satırı benzersiz tanımlar.

Örnek:

```
sporcu_id BIGSERIAL PRIMARY KEY
```

Bu sayede her sporcu için benzersiz bir sporcu_id oluşur.

6.4 Foreign Key

Foreign key, bir tabloyu başka tabloya bağlar.

Örnek:

```
sporcu_id BIGINT NOT NULL REFERENCES sporcu(sporcu_id)
```

Bu şu demektir:

uyelik tablosundaki sporcu_id değeri, sporcu tablosunda gerçekten var olmalıdır.

Bu yapı ilişkisel veritabanının temelidir.

6.5 CHECK Constraint

CHECK, belirli bir alanın hangi şartlara uyması gerektiğini söyler.

Örnek:

```
CHECK (bitis_tarihi > baslangic_tarihi)
```

Bu kural, üyelik bitiş tarihinin başlangıç tarihinden sonra olmasını zorunlu kılar.

Başka bir örnek:

```
rol VARCHAR(20) NOT NULL CHECK (rol IN ('PT', 'DIYETISYEN', 'DIGER'))
```

Bu da rol alanının sadece üç değerden birini almasını sağlar.

6.6 UNIQUE Constraint

UNIQUE, aynı değerin iki kez yazılmasını engeller.

Örnek:

```
telefon VARCHAR(20) NOT NULL UNIQUE
```

Bu sayede aynı telefon numarasıyla iki sporcu kaydı oluşturulamaz.

6.7 Index

Index, veritabanı sorgularının daha hızlı çalışmasını sağlar. Bu projede ayrıca bir iş kuralını uygulamak için de kullanılmıştır.

Önemli örnek:

```
CREATE UNIQUE INDEX uq_sporcu_tek_aktif_uyelik  
ON uyelik(sporcu_id)  
WHERE durum = 'AKTIF';
```

Bu bir partial unique index'tir.

Normal unique index olsaydı, bir sporcunun geçmişteki tüm üyelikleri de engellenirdi. Ama burada sadece durum = 'AKTIF' olan kayıtlar için benzersizlik aranır.

Sonuç:

```
1  
Ayn sporcu şgeçmite birçok üyelik ıkaydna sahip olabilir,  
ama ıayn anda ıyalnzca bir aktif güyelie sahip olabilir.
```

Bölüm 7

Trigger ve Function Mantığı

Trigger, veritabanında belirli bir işlem olduğunda otomatik çalışan yapıdır.

Örneğin:

```
uyelik tablosuna ıkayt eklenmeden önce ıtutar otomatik hesapla
```

Bu iş için önce function yazılır, sonra trigger bu function'a bağlanır.

7.1 Function Syntax

PostgreSQL function örneği:

```
CREATE OR REPLACE FUNCTION trg_uyelik_tutari_hesapla()  
RETURNS TRIGGER AS $$  
DECLARE  
    hesaplanan_tutar NUMERIC(10,2);  
BEGIN  
    -- işlem yapılr  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Burada:

Syntax	Anlamı
CREATE OR REPLACE FUNCTION	Function yoksa oluşturur, varsa günceller.
RETURNS TRIGGER	Bu function trigger tarafından çalıştırılır.
DECLARE	Değişken tanımlama bölümüdür.
BEGIN ... END	Function'ın çalışacak kod bloğudur.
NEW	Eklenen veya güncellenen yeni satırı temsil eder.
LANGUAGE plpgsql	PostgreSQL'in prosedürel dilini kullanır.

7.2 Üyelik Tutarı Trigger'ı

Üyelik tutarı şu formülle hesaplanır:

```
plan.aylik_ucret * uyelik_suresi.katsayi
```

İlgili trigger:

```
CREATE TRIGGER uyelik_tutari_hesapla  
BEFORE INSERT OR UPDATE OF plan_id, sure_id, tutar ON uyelik  
FOR EACH ROW  
EXECUTE FUNCTION trg_uyelik_tutari_hesapla();
```

Bu şu anlama gelir:

- uyelik tablosuna kayıt eklenmeden önce çalış.
- plan_id, sure_id veya tutar güncellenirse de çalış.
- Her satır için ayrı çalış.
- trg_uyelik_tutari_hesapla() function'ını çağır.

Bu sayede uygulama yanlış tutar göndermeye çalışsa bile veritabanı doğru tutarı yazar.

7.3 Atama Kontrol Trigger'ı

sporcu_atama tablosunda PT ve diyetisyen kontrolü yapılır.

Örneğin:

- PT alanına sadece rol = 'PT' olan çalışan gelebilir.
- Diyetisyen alanına sadece rol = 'DIYETISYEN' olan çalışan gelebilir.

Bu kontrol veritabanı seviyesinde yapılır. Böylece uygulama yanlış çalışan ID'si gönderse bile PostgreSQL bunu kabul etmez.

7.4 DEFERRABLE Trigger

Bu projedeki en önemli detaylardan biri şudur:

```
DEFERRABLE INITIALLY DEFERRED
```

Bu ifade, trigger kontrolünün transaction sonuna ertelenmesini sağlar.

Neden gerekli?

Üyelik oluştururken iki kayıt girilir:

1. uyelik tablosuna kayıt
2. sporcu_atama tablosuna kayıt

Profesyonel plan için diyetisyen zorunludur. Eğer trigger üyelik eklenir eklenmez kontrol yapsaydı, atama kaydı henüz eklenmediği için hata oluşurdu.

Bu yüzden kontrol transaction sonuna ertelenir.

Akış:

```
BEGIN
  üyelik ekle
  sporcu_atama ekle
COMMIT ıannda kontrol et
```

Bu doğru ve temiz bir veritabanı tasarımıdır.

Bölüm 8

seed.sql Dosyası

seed.sql, demo verileri yükler.

Bu dosya sayesinde uygulama boş açılmaz. Sunumda gösterebilmek için hazır sporcular, çalışanlar, üyelikler ve ödemeler gelir.

Örnek:

```
INSERT INTO plan (plan_id, plan_adi, aylık_ucret) VALUES
  (1, 'Temel', 500.00),
  (2, 'Profesyonel', 1500.00),
  (3, 'VIP', 2500.00);
```

Bu satırlar üç üyelik planını oluşturur.

Süreler:

```
INSERT INTO üyelik_suresi (sure_id, ay, katsayı) VALUES
  (1, 1, 1.00),
  (2, 3, 2.50),
  (3, 6, 5.00),
  (4, 12, 9.00);
```

Burada 12 aylık üyelikte katsayı 9'dur. Yani 12 ay VIP üyeliğin fiyatı:

$2500 * 9 = 22500$

Seed veride farklı durumlar özellikle eklenmiştir:

- Aktif üyelikler
- Bitmiş üyelik
- Yakında bitecek üyelikler
- Temel, Profesyonel ve VIP planlar
- Farklı ödeme yöntemleri
- PT ve diyetisyen atamaları

Bu sayede dashboard ve sorgular ekranı anlamlı veri gösterir.

Bölüm 9

db.py Dosyası

db.py, veritabanı bağlantı işlemlerini merkezileştirir.

Amaç şudur:

Her route içinde tekrar tekrar bağlantı kodu yazmak yerine, ortak helper fonksiyonlar kullanmak.

9.1 load_dotenv

```
from dotenv import load_dotenv

load_dotenv()
```

Bu kod .env dosyasını okur. Böylece DATABASE_URL gibi değerler Python içinde kullanılabilir.

9.2 get_database_url

```
def get_database_url():
    url = os.getenv("DATABASE_URL")
    if not url:
        raise RuntimeError("DATABASE_URL .env dosyasında tanımlı değil.")
    return url
```

Bu fonksiyon .env içindeki veritabanı bağlantı adresini alır.

Eğer DATABASE_URL yoksa hata verir. Bu iyi bir pratiktir çünkü veritabanı olmadan uygulama sağlıklı çalışamaz.

9.3 get_conn

```
def get_conn():
    return psycopg2.connect(get_database_url(), row_factory=dict_row)
```

Bu fonksiyon PostgreSQL bağlantısı oluşturur.

row_factory=dict_row önemlidir. Normalde veritabanından gelen satırlar tuple gibi döner. dict_row kullanınca alan adlarıyla erişebiliriz:

```
row["ad"]  
row["soyad"]
```

Bu, template ve route kodlarını daha okunur yapar.

9.4 transaction

```
@contextmanager  
def transaction():  
    with get_conn() as conn:  
        try:  
            yield conn  
            conn.commit()  
        except Exception:  
            conn.rollback()  
            raise
```

Transaction, birden fazla veritabanı işlemini tek paket gibi çalıştırır.

Örneğin üyelik oluştururken:

1. Üyelik eklenir.
2. Atama eklenir.
3. Log kaydı eklenir.

Bunlardan biri hata verirse hepsi geri alınır.

Bu yüzden transaction veri tutarlılığı için önemlidir.

9.5 fetch_all ve fetch_one

```
def fetch_all(sql, params=None):  
    ...  
    cur.execute(sql, params or ())  
    return cur.fetchall()
```

fetch_all, sorgudan gelen tüm satırları döndürür.

```
def fetch_one(sql, params=None):  
    ...  
    cur.execute(sql, params or ())  
    return cur.fetchone()
```

fetch_one, tek satır döndürür.

Örnek:

```
admin = fetch_one(  
    "SELECT * FROM yonetici WHERE email = %s",  
    (email,) )
```

Buradaki %s, SQL parametre yer tutucusudur. Kullanıcıdan gelen veriyi doğrudan string içine yazmak yerine parametre olarak vermek SQL injection riskini azaltır.

Bölüm 10

app.py Dosyası

app.py, uygulamanın ana dosyasıdır.

Bu dosyada:

- Flask app oluşturulur.
- Login sistemi kurulur.
- Route'lar tanımlanır.
- Form verileri işlenir.
- SQL sorguları çağrılır.
- Template'ler render edilir.
- Yönetici logları yazılır.

10.1 Import Bölümü

Dosyanın başında şu tür importlar bulunur:

```
from flask import Flask, flash, jsonify, redirect, render_template, request, session, url_for
from werkzeug.security import check_password_hash
from db import fetch_all, fetch_one, get_conn, transaction
```

Anlamları:

Import	Görev
Flask	Uygulama nesnesini oluşturur.
render_template	HTML/Jinja dosyasını render eder.
request	Kullanıcıdan gelen form verilerini okur.
session	Giriş yapan kullanıcı bilgisini tutar.
redirect	Kullanıcıyı başka sayfaya yönlendirir.
url_for	Route adına göre URL üretir.
flash	Kullanıcıya başarı/hata mesajı gösterir.

jsonify	JSON API cevabı döndürür.
check_password_hash	Hashlenmiş şifreyi kontrol eder.

10.2 Flask App Oluşturma

```
app = Flask(__name__)
app.secret_key = os.getenv("SECRET_KEY", "dev-secret-change-me")
```

Flask(__name__), Flask uygulamasını oluşturur.
secret_key, session ve flash mesajları için gereklidir.

10.3 Decorator Nedir?

Python'da @ ile başlayan yapılara decorator denir.

Flask route örneği:

```
@app.get("/dashboard")
def dashboard():
    ...
```

Bu, Flask'a şunu söyler:

/dashboard adresine GET isteği gelirse dashboard fonksiyonunu ısıçaltr.

10.4 login_required Decorator'ı

```
def login_required(view):
    @wraps(view)
    def wrapped_view(*args, **kwargs):
        if not session.get("yonetici_id"):
            return redirect(url_for("login"))
        return view(*args, **kwargs)

    return wrapped_view
```

Bu decorator, sayfaya girmeden önce kullanıcının login olup olmadığını kontrol eder.

Örneğin:

```
@app.get("/dashboard")
@login_required
def dashboard():
    ...
```

Bu sayede giriş yapmayan kullanıcı dashboard'a gidemez, login sayfasına yönlendirilir.

10.5 Template Filter

Template filter, Python tarafındaki bir fonksiyonu Jinja içinde kullanılabilir hale getirir.

Örnek:

```
@app.template_filter("money")
def money(value):
    ...
```

Template içinde:

```
{{ stats.aylik_gelir|money }}
```

Bu, sayısal değeri para formatında gösterir.

Örneğin:

```
1500.00 -> 1.500,00 TL
```

10.6 Login Route'u

Login route'u:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    ...
```

Burada hem GET hem POST kabul edilir.

GET:

Login formunu gösterir.

POST:

```
111
Kullancnn göğönderdii email ve şifreyi kontrol eder.
```

Login akışı:

1. Email ve şifre formdan alınır.
2. yönetici tablosunda email aranır.
3. Hashlenmiş şifre check_password_hash ile kontrol edilir.
4. Başarılysa session içine yönetici bilgisi yazılır.
5. LOGIN_SUCCESS logu oluşturulur.
6. Dashboard'a yönlendirilir.

Başarısızsa:

1. LOGIN_FAIL logu yazılır.
2. Hata mesajı gösterilir.

10.7 Dashboard Route'u

Dashboard route'u özet metrikleri hazırlar:

```
@app.get("/dashboard")
@login_required
def dashboard():
    stats = fetch_one(...)
    ending_memberships = fetch_all(...)
    return render_template("dashboard.html", stats=stats, ending_memberships=
ending_memberships)
```

Burada:

- stats: kartlarda gösterilen sayısal özetlerdir.
- ending_memberships: 7 gün içinde bitecek üyelik listesidir.

10.8 JSON API Route'ları

Chart.js grafiklerinin verileri API route'larından gelir.

Örnek:

```
@app.get("/api/dashboard/plan-dagilimi")
@login_required
def api_plan_distribution():
    rows = fetch_all(...)
    return jsonify({
        "labels": [...],
        "values": [...]
    })
```

Bu route HTML değil, JSON döndürür.

Örnek JSON:

```
{
  "labels": ["Profesyonel", "Temel", "VIP"],
  "values": [2, 2, 2]
}
```

JavaScript bu veriyi alır ve grafiğe dönüştürür.

10.9 CRUD Nedir?

CRUD, temel veri işlemleridir:

Harf	Anlam	Projedeki örnek
C	Create	Sporcu ekleme

R	Read	Sporcu listeleme
U	Update	Sporcu düzenleme
D	Delete	Pasife alma

Projede fiziksel silme yerine genellikle pasife alma kullanılır. Bu daha güvenlidir çünkü geçmiş veriler kaybolmaz.

Bölüm 11

Route'ların Görevleri

Projede route'lar kullanıcı ekranlarına karşılık gelir.

Route	Görev
/login	Yönetici giriş ekranı
/dashboard	Ana özet ekranı
/sporcular	Sporcu listeleme
/sporcular/yeni	Sporcu ekleme
/sporcular/<id>/duzenle	Sporcu düzenleme
/calisanlar	Çalışan listeleme
/calisanlar/yeni	Çalışan ekleme
/uyelikler/yeni	Üyelik oluşturma
/uyelikler/<id>	Üyelik detayı
/odemeler	Ödeme listesi ve borç özeti
/odemeler/yeni	Ödeme ekleme
/sorgular	Hazır SQL sorgu sonuçları
/loglar	Yönetici logları

Bu yapı, sunum sırasında da kolay anlatılır:

Önce şgiri ııyapıyor, dashboard görölüyor, sonra sporcu/üyelik/ödeme şilemleri ııyapıyor.

Bölüm 12

Üyelik Oluşturma Akışı

Üyelik oluşturma projenin en önemli akışıdır.

Kullanıcı formda şunları seçer:

- Sporcu
- Plan
- Süre
- Başlangıç tarihi
- Diyetisyen
- PT

Plan kuralları:

Plan	Kural
Temel	Atama zorunlu değil.
Profesyonel	Diyetisyen zorunlu.
VIP	Diyetisyen ve PT zorunlu.

Python tarafında önce uygulama kontrolü yapılır:

```
if plan["plan_adi"] in ("Profesyonel", "VIP") and not diyetisyen_id:  
    raise ValueError("Diyetisyen seçilmelidir.")
```

Sonra veritabanı da aynı kuralı trigger ile korur.

Bu iki katmanlı güvenlidir:

```
Uygulama ıkatman kontrol eder.1  
Veritaban ıkatman son güvenlik olarak kontrol eder.
```

Bölüm 13

Ödeme Akışı

Ödeme ekranında aktif üyelikler listelenir. Her üyelik için:

- Toplam tutar
- Ödenen tutar
- Kalan borç

hesaplanır.

Kalan borç SQL ile hesaplanır:

```
u.tutar - COALESCE(SUM(o.tutar), 0) AS kalan
```

Buradaki COALESCE önemlidir.

Eğer hiç ödeme yoksa `SUM(o.tutar)` sonucu `NULL` olabilir. `COALESCE(..., 0)` bunu sıfıra çevirir.

Yani:

```
Ödeme yoksa ödenen = 0 kabul edilir.
```

Bölüm 14

Yönetici Loglama

Projede önemli işlemler `yonetici_log` tablosuna kaydedilir.

Loglanan olay örnekleri:

- LOGIN_SUCCESS
- LOGIN_FAIL
- LOGOUT
- CREATE_SPORCU
- UPDATE_SPORCU
- DEACTIVATE_SPORCU
- CREATE_CALISAN
- UPDATE_CALISAN
- CREATE_UYELIK
- CREATE_ODEME

Log ekleyen fonksiyon:

```
def log_event(event_type, detail=None, yonetici_id=None, conn=None):  
    ...
```

Bu fonksiyon şu bilgileri kaydeder:

- Yönetici ID
- Olay tipi
- Detay
- IP adresi
- User agent
- Olay zamanı

Bu sayede sistemde kimin ne yaptığı takip edilebilir.

Bölüm 15

templates Klasörü

templates/ klasörü HTML/Jinja dosyalarını içerir.

Flask'ta `render_template("dosya.html")` dendiğinde Flask bu klasöre bakar.

15.1 base.html

base.html, tüm sayfaların ortak iskeletidir.

İçinde:

- HTML başlangıcı
- CSS linki
- Sidebar menü
- Topbar
- Flash mesaj alanı
- İçerik bloğu
- Script bloğu

bulunur.

Önemli Jinja yapısı:

```
{% block content %}{% endblock %}
```

Diğer template'ler bu alanı doldurur.

15.2 extends Kullanımı

Örneğin dashboard.html şununla başlar:

```
{% extends "base.html" %}
```

Bu şu demektir:

Bu sayfa base.html iskeletini kullanacak.

Sonra sayfaya özel içerik şu bloklara yazılır:

```
{% block page_title %}Dashboard{% endblock %}
{% block content %}
    ...
{% endblock %}
```

Bu sayede her sayfada aynı sidebar/topbar kodunu tekrar yazmak gerekmez.

15.3 Jinja Değişkenleri

Jinja değişkenleri çift süslü parantezle yazılır:

```
{{ stats.aktif_sporcu }}
```

Bu, Python'dan gelen stats içindeki aktif_sporcu alanını ekrana basar.

15.4 Jinja Döngüleri

Liste verileri için:

```
{% for row in ending_memberships %}
    <tr>
        <td>{{ row.ad }} {{ row.soyad }}</td>
    </tr>
{% else %}
    <tr><td>Veri yok.</td></tr>
{% endfor %}
```

{% else %} kısmı liste boşsa çalışır.

15.5 Jinja Koşulları

Koşul örneği:

```
{% if sporcu.aktif_mi %}
    Aktif
{% else %}
    Pasif
{% endif %}
```

Bu, veriye göre farklı HTML göstermeyi sağlar.

Bölüm 16

static/css/app.css Dosyası

app.css, uygulamanın tüm görsel tasarımını içerir.

16.1 CSS Değişkenleri

Dosyanın başında :root içinde renk ve ölçü değişkenleri vardır:

```
:root {
  --bg: #f4f6f8;
  --surface: #ffffff;
  --text: #17212b;
  --primary: #126b5f;
  --radius: 8px;
}
```

Bu değişkenler daha sonra şöyle kullanılır:

```
body {
  background: var(--bg);
  color: var(--text);
}
```

Avantajı:

Rengi tek yerden şşdeitirince tüm arayüz güncellenir.

16.2 Layout

Sidebar sabit konumda durur:

```
.sidebar {
  position: fixed;
  width: 248px;
}
```

Ana içerik sidebar kadar soldan boşluk bırakır:

```
.main {
  margin-left: 248px;
}
```

Bu sayede sol menü ve ana içerik yan yana görünür.

16.3 Grid Kullanımı

Dashboard kartları grid ile dizilir:

```
.metric-grid {  
  display: grid;  
  grid-template-columns: repeat(4, minmax(0, 1fr));  
  gap: 14px;  
}
```

Bu şu demektir:

4 şeit sütun şoludur, ıaralarında 14px şboluk ıbrak.

16.4 Responsive Tasarım

Mobil uyumluluk için media query kullanılır:

```
@media (max-width: 980px) {  
  .sidebar {  
    position: static;  
    width: auto;  
  }  
}
```

Bu şu anlama gelir:

Ekran 980px'den küçükse sidebar ıartk sabit ıdurmasn, normal ışaka girsin.

Bu sayede uygulama mobilde de kullanılabilir.

Bölüm 17

static/js/dashboard.js Dosyası

Bu dosya dashboard grafiklerini oluşturur.

17.1 fetch

JavaScript'te fetch, bir URL'den veri almak için kullanılır.

```
async function loadJson(url) {  
    const response = await fetch(url);  
    return response.json();  
}
```

Bu fonksiyon:

1. Verilen URL'ye istek atar.
2. Cevabı JSON'a çevirir.
3. JavaScript objesi olarak döndürür.

17.2 Promise.all

Dashboard iki farklı API'den veri alır:

```
Promise.all([  
    loadJson("/api/dashboard/plan-dagilimi"),  
    loadJson("/api/dashboard/aylik-gelir")  
])
```

Promise.all, iki isteğin de bitmesini bekler.

17.3 Chart.js

Plan dağılımı için doughnut grafik:

```
new Chart(planCtx, {  
    type: "doughnut",  
    data: {  
        labels: planData.labels,  
        datasets: [{
```

```
        data: planData.values
    }]
  }
});
```

Gelir için bar grafik:

```
new Chart(incomeCtx, {
  type: "bar",
  data: {
    labels: incomeData.labels,
    datasets: [{
      label: "Gelir",
      data: incomeData.values
    }]
  }
});
```

Burada:

Alan	Anlam
type	Grafik türü
labels	X eksenini veya kategori adları
datasets	Grafikte gösterilecek sayısal veriler
data	Değerler

Bölüm 18

SQL Sorgu Mantığı

Projede birçok SQL sorgusu vardır. Bunlar hem ekranda veri göstermek hem de rapor üretmek için kullanılır.

18.1 SELECT

Veri çekmek için:

```
SELECT ad, soyad FROM sporcu;
```

18.2 WHERE

Filtrelemek için:

```
SELECT * FROM uyelik  
WHERE durum = 'AKTIF';
```

18.3 JOIN

Tabloları birleştirmek için:

```
SELECT s.ad, s.soyad, p.plan_adi  
FROM uyelik u  
JOIN sporcu s ON s.sporcu_id = u.sporcu_id  
JOIN plan p ON p.plan_id = u.plan_id;
```

Bu sorgu üyelik tablosunu sporcu ve plan tablolarıyla birleştirir.

18.4 LEFT JOIN

LEFT JOIN, soldaki tablodaki kayıtları her durumda getirir.

Ödeme sorgusunda kullanılır çünkü bazı üyeliklerin henüz ödemesi olmayabilir.

```
LEFT JOIN ödeme o ON o.uyelik_id = u.uyelik_id
```

18.5 GROUP BY

Gruplama yapmak için kullanılır.

Örneğin plan dağılımı:

```
SELECT p.plan_adi, COUNT(*) AS adet
FROM uyelik u
JOIN plan p ON p.plan_id = u.plan_id
WHERE u.durum = 'AKTIF'
GROUP BY p.plan_adi;
```

Bu sorgu her plan türünde kaç aktif üyelik olduğunu hesaplar.

18.6 Aggregate Fonksiyonlar

Aggregate fonksiyonlar birden fazla satırdan tek sonuç üretir.

Fonksiyon	Görev
COUNT(*)	Satır sayar.
SUM(tutar)	Toplam alır.
AVG(maas)	Ortalama alır.
MIN(...)	En küçük değeri bulur.
MAX(...)	En büyük değeri bulur.

18.7 COALESCE

COALESCE, NULL değer yerine başka değer kullanır.

Örnek:

```
COALESCE(SUM(o.tutar), 0)
```

Eğer ödeme yoksa SUM(o.tutar) sonucu NULL olur. Bu ifade onu 0 yapar.

Bölüm 19

Dashboard Metrikleri

Dashboard ekranında dört ana kart vardır:

- Aktif sporcu sayısı
- Bu ay gelir
- 7 günde bitecek üyelik sayısı
- Çalışan sayısı

Bu metrikler SQL ile hesaplanır.

Örneğin bu ay gelir:

```
SELECT COALESCE(SUM(tutar), 0)
FROM odeme
WHERE date_trunc('month', odeme_tarihi) = date_trunc('month', CURRENT_DATE);
```

`date_trunc('month', ...)` tarihin ay seviyesine indirilmesini sağlar.

Yani:

```
2026-05-24 -> 2026-05-01
```

Bu sayede aynı ay içindeki ödemeler bulunur.

Bölüm 20

Sorgular Ekranı

/sorgular ekranı, veritabanı projesi için özellikle önemlidir. Çünkü burada hazır SQL raporları gösterilir.

Ekrandaki raporlar:

- 7 gün içinde bitecek üyelikler
- PT başına aktif sporcu sayısı
- Diyetisyen başına aktif sporcu sayısı
- Aylık toplam gelir
- Plan türüne göre dağılım
- Kalan gün bilgisiyle aktif üyelikler

Bu ekran sunumda SQL bilginizi göstermek için güçlüdür.

Bölüm 21

İş Kuralları

Projede iş kuralları hem uygulama hem veritabanı seviyesinde uygulanır.

21.1 Tek Aktif Üyelik

Kural:

```
1
Ayn sporcunun ıayn anda sadece bir aktif üyeli olabilir.
```

Veritabanı çözümü:

```
CREATE UNIQUE INDEX uq_sporcu_tek_aktif_uyelik
ON uyelik(sporcu_id)
WHERE durum = 'AKTIF';
```

21.2 Profesyonel Plan

Kural:

```
Profesyonel plan için diyetisyen zorunludur.
```

Hem uygulama hem trigger tarafından kontrol edilir.

21.3 VIP Plan

Kural:

```
VIP plan için hem diyetisyen hem PT zorunludur.
```

Bu kural da hem uygulama hem trigger tarafından kontrol edilir.

21.4 Ücret Hesaplama

Kural:

```
Üyelik ıtutar = plan ıaylk ücreti * süre ııkatsays
```

Örnek:

VIP 12 ay = 2500 * 9 = 22500

Bölüm 22

Güvenlik ve Doğru Uygulama Notları

Bu proje temel seviyede bazı doğru güvenlik pratiklerini kullanır.

22.1 Şifre Hashleme

Yönetici şifresi düz metin saklanmaz. Hashlenmiş halde `yonetici.sifre_hash` alanında tutulur.

Kontrol:

```
check_password_hash(admin["sifre_hash"], password)
```

Bu, girilen şifreyi hash ile karşılaştırır.

22.2 SQL Parametreleri

Sorgularda kullanıcı verisi doğrudan string içine gömülmez.

Doğru kullanım:

```
cur.execute("SELECT * FROM yonetici WHERE email = %s", (email,))
```

Bu yöntem SQL injection riskini azaltır.

Yanlış kullanım:

```
cur.execute(f"SELECT * FROM yonetici WHERE email = '{email}'")
```

Bu güvenli değildir.

22.3 Session

Login başarılı olunca:

```
session["yonetici_id"] = admin["yonetici_id"]
```

Bu bilgi kullanıcının oturumunda tutulur. Böylece giriş yapmış kullanıcı dashboard ve diğer ekranlara erişebilir.

Bölüm 23

Uygulamayı Çalıştırma

Proje klasörüne gir:

```
cd "/home/baranbeey/Masaüstü/Baran/Okul/4. 11Yaryl/1Veritaban/spor_salonu_app"
```

Sanal ortamı aktif et:

```
source .venv/bin/activate
```

Gerekirse paketleri yükle:

```
pip install -r requirements.txt
```

Flask sunucusunu başlat:

```
flask --app app run --debug
```

Tarayıcıdan aç:

```
http://127.0.0.1:5000
```

Demo giriş:

```
Email: admin@spor.local$  
ifre: admin123
```

Bölüm 24

Sunum İçin Anlatım Akışı

Sunumda şu sırayla anlatmak iyi olur:

1. Proje konusu: spor salonu yönetim sistemi.
2. Kullanılan teknolojiler: Flask, PostgreSQL, HTML/CSS, Chart.js.
3. ER tasarımından gelen tablolar.
4. `schema.sql` içinde primary key, foreign key, check constraint ve index yapıları.
5. Trigger ile üyelik tutarı ve atama kuralları.
6. Uygulamaya yönetici olarak giriş.
7. Dashboard metrikleri ve grafikler.
8. Sporcu listeleme ve yeni sporcu ekleme.
9. Üyelik oluşturma ve Profesyonel/VIP kurallarını gösterme.
10. Ödeme ekleme ve kalan borç hesabı.
11. Sorgular ekranında SQL raporlarını gösterme.
12. Loglar ekranında işlemlerin kaydedildiğini gösterme.

Bölüm 25

Dosya Dosya Kısa Özet

Dosya	Görev
app.py	Flask route'ları ve uygulama mantığı
db.py	PostgreSQL bağlantı yardımcıları
schema.sql	Veritabanı yapısı
seed.sql	Demo veri
requirements.txt	Python paketleri
.env	Ortam değişkenleri
templates/base.html	Ortak HTML iskeleti
templates/login.html	Giriş ekranı
templates/dashboard.html	Dashboard ekranı
templates/sporcular.html	Sporcu listeleme
templates/uyelik_form.html	Üyelik oluşturma formu
templates/odemeler.html	Ödeme ve borç ekranı
templates/sorgular.html	SQL rapor ekranı
templates/loglar.html	Yönetici log ekranı
static/css/app.css	Tasarım
static/js/dashboard.js	Grafikler
sql_islemleri_raporu.md	SQL raporu
tutorial.md	Bu öğretici doküman

Bölüm 26

Bu Projeden Öğrenilen Ana Konular

Bu proje üzerinden şu konular öğrenilir:

- Flask ile route yazma
- HTML template render etme
- Jinja syntax kullanma
- PostgreSQL tablo tasarlama
- Primary key ve foreign key ilişkileri kurma
- Check constraint ve unique constraint kullanma
- Partial unique index ile özel iş kuralı yazma
- Trigger ve function mantığı
- Python ile PostgreSQL bağlantısı
- Transaction kullanımı
- Form verisi işleme
- Session ile login kontrolü
- Chart.js ile grafik çizme
- CSS ile yönetim paneli tasarlama
- SQL rapor sorguları yazma

Bölüm 27

HTTP, Request ve Response Mantığı

Web uygulamalarını anlamak için HTTP mantığını bilmek önemlidir. Tarayıcı ile Flask arasındaki iletişim HTTP istekleriyle yapılır.

Bu projede en çok iki HTTP yöntemi kullanılır:

Yöntem	Anlamı	Projedeki kullanım
GET	Sayfa veya veri istemek için kullanılır.	Listeleme, dashboard, form gösterme
POST	Form verisi göndermek için kullanılır.	Login, sporcu ekleme, ödeme ekleme

Örneğin login route'u:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    ...
```

Bu route iki şekilde çalışır:

```
GET /login -> şgiri formunu gösterir
POST /login -> formdan gelen email ve şifreyi kontrol eder
```

Flask'ta formdan gelen veri `request.form` ile okunur.

Örnek:

```
email = request.form.get("email", "").strip().lower()
password = request.form.get("password", "")
```

Burada:

Kod	Anlamı
<code>request.form</code>	POST ile gelen form verilerini tutar.
<code>.get("email", "")</code>	email alanını alır, yoksa boş string döndürür.

<code>.strip()</code>	Başta ve sondaki boşlukları siler.
<code>.lower()</code>	Email'i küçük harfe çevirir.

Flask bir route sonunda genellikle response döndürür.

Örnek response tipleri:

```
return render_template("dashboard.html")
return redirect(url_for("login"))
return jsonify({"labels": labels, "values": values})
```

Bu üç satırın farkı:

Fonksiyon	Dönen şey
<code>render_template</code>	HTML sayfası
<code>redirect</code>	Başka URL'ye yönlendirme
<code>jsonify</code>	JSON API cevabı

Bölüm 28

Route, Template ve SQL Eşleştirme Tablosu

Bu projeyi anlamanın en iyi yollarından biri, her route'un hangi template'i kullandığını ve hangi veritabanı işlemini yaptığını görmektir.

Route	Python fonksiyonu	Template	Ana SQL işlemi
/login	login()	login.html	yonetici sorgusu, yonetici_log insert
/dashboard	dashboard()	dashboard.html	Metrik ve yaklaşan üyelik sorguları
/api/dashboard/plan-dagilimi	api_plan_distribution()	JSON	Plan dağılımı sorgusu
/api/dashboard/aylik-gelir	api_monthly_income()	JSON	Son 6 ay gelir sorgusu
/sporcular	sporcular()	sporcular.html	Sporcu listeleme ve aktif üyelik bilgisi
/sporcular/yeni	sporcu_yeni()	sporcu_form.html	INSERT INTO sporcu
/sporcular/<id>/duzenle	sporcu_duzenle()	sporcu_form.html	UPDATE sporcu
/calisanlar	calisanlar()	calisanlar.html	Çalışan listeleme
/calisanlar/yeni	calisan_yeni()	calisan_form.html	INSERT INTO calisan
/uyelikler/yeni	uyelik_yeni()	uyelik_form.html	INSERT INTO uyelik, INSERT INTO sporcu_atama
/uyelikler/<id>	uyelik_detay()	uyelik_detay.html	Üyelik, atama ve ödeme join sorgusu

/odemeler	odemeler()	odemeler.html	Ödeme listesi ve kalan borç sorgusu
/odemeler/yeni	odeme_yeni()	odeme_form.html	INSERT INTO odeme
/sorgular	sorgular()	sorgular.html	Hazır rapor sorguları
/loglar	loglar()	loglar.html	Yönetici log listeleme

Bu tablo sunumda çok işe yarar. Çünkü uygulamanın her ekranının arkasında hangi SQL işleminin çalıştığını doğrudan gösterir.

Bölüm 29

Werkzeug ve Şifre Hashleme

Projede yönetici şifresi düz metin saklanmaz. Bu işlem için Flask ekosisteminde sık kullanılan Werkzeug kütüphanesinin güvenlik fonksiyonu kullanılır.

app.py içinde:

```
from werkzeug.security import check_password_hash
```

Login sırasında:

```
if admin and check_password_hash(admin["sifre_hash"], password):  
    ...
```

Bu kodun mantığı:

1. Kullanıcı formdan şifre girer.
2. Veritabanındaki sifre_hash alınır.
3. Girilen şifre hash ile karşılaştırılır.
4. Eşleşirse login başarılı olur.

Buradaki önemli nokta şudur:

```
1  
Veritabanında admin123 gibi düz metin şifre tutulmaz.
```

seed.sql içinde admin kullanıcısı hashlenmiş şifreyle eklenmiştir:

```
INSERT INTOyonetici (... , sifre_hash, ...)  
VALUES (... , 'scrypt:... ', ...);
```

Hash değeri tek yönlüdür. Yani hash'ten şifrenin kendisini doğrudan geri elde etmek amaçlanmaz. Login kontrolünde girilen şifre aynı algoritmayla kontrol edilir.

Bölüm 30

Flash Mesajları ve Hata Gösterimi

Flask'ta flash, kullanıcıya geçici mesaj göstermek için kullanılır.

Örneğin sporcu başarıyla oluşturulunca:

```
flash("Sporcu şoluduruldu.", "success")
```

Hata olursa:

```
flash(message, "error")
```

Bu mesajlar base.html içinde gösterilir:

```
{% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="flash {{ category }}">{{ message }}</div>
    {% endfor %}
  {% endif %}
{% endwith %}
```

Burada:

Jinja parçası	Anlamı
get_flashed_messages	Flask'ın sakladığı geçici mesajları alır.
with_categories=true	Mesajla birlikte kategori de döner.
category	success veya error gibi CSS sınıfı olur.
message	Kullanıcıya gösterilecek metindir.

Bu sayede aynı HTML yapısı başarı ve hata mesajlarını farklı renklerle gösterebilir.

Projede veritabanı hatalarını sadeleştirmek için şu yardımcı fonksiyon vardır:

```
def safe_flash_db_error(error):
    message = str(error).splitlines()[0]
    flash(message, "error")
```

PostgreSQL bazen çok satırlı hata mesajı döndürür. Bu fonksiyon ilk satırı alıp kullanıcıya daha temiz bir mesaj gösterir.

Bölüm 31

seed.sql İçindeki Transaction ve setval Mantığı

seed.sql dosyası şu şekilde başlar ve biter:

```
BEGIN;  
...  
COMMIT;
```

Bu yapı transaction başlatır.

Anlamı:

Demo veriyi tek şilem paketi olarak yükle.
Bir yerde hata olursa tüm yükleme geri alınır.

Bu önemlidir çünkü seed sırasında planlar, süreler, çalışanlar, sporcular, üyelikler, atamalar ve ödemeler birbirine bağlı şekilde eklenir.

Dosyanın sonunda şu satırlar vardır:

```
SELECT setval('calisan_calisan_id_seq', (SELECT MAX(calisan_id) FROM calisan)  
);  
SELECT setval('sporcu_sporcu_id_seq', (SELECT MAX(sporcu_id) FROM sporcu));  
SELECT setval('uyelik_uyelik_id_seq', (SELECT MAX(uyelik_id) FROM uyelik));  
SELECT setval('yonetici_yonetici_id_seq', (SELECT MAX(yonetici_id) FROM  
yonetici));
```

BIGSERIAL kolonlar PostgreSQL'de otomatik artan sequence yapısı kullanır. Seed dosyasında bazı ID'ler elle verilmiştir:

```
INSERT INTO sporcu (sporcu_id, ad, soyad, ...)  
VALUES (1, 'Ali', 'Yildiz', ...);
```

Elle ID verildiğinde sequence bazen geride kalabilir. setval, sequence değerini tablodaki en büyük ID'ye ayarlar.

Bu yapılmazsa yeni kayıt eklerken PostgreSQL tekrar 1, 2, 3 gibi daha önce kullanılmış ID üretmeye çalışabilir ve primary key hatası oluşabilir.

Kısaca:

setval, demo veriden sonra otomatik ID üretimini doğru yerden devam ettirir.

Bölüm 32

psql Komut Satırı Syntax'ı

psql, PostgreSQL'in komut satırı aracıdır. Bu projede veritabanı kurmak ve SQL dosyalarını çalıştırmak için kullanılır.

Bağlantı örneği:

```
psql "postgresql://spor_app:spor_app_123@localhost:5432/spor_salonu_db"
```

Dosya çalıştırma örneği:

```
psql "postgresql://spor_app:spor_app_123@localhost:5432/spor_salonu_db" -f  
schema.sql
```

Burada:

Parça	Anlamı
psql	PostgreSQL komut satırı istemcisi
Bağlantı URL'i	Hangi kullanıcıyla hangi veritabanına bağlanılacağını söyler
-f schema.sql	SQL komutlarını dosyadan oku ve çalıştır

psql içindeki bazı özel komutlar SQL değildir, psql komutudur:

Komut	Görev
\c spor_salonu_db	Belirtilen veritabanına geçer.
\q	psql'den çıkar.
\dt	Tabloları listeler.
\d tablo_adi	Bir tablonun yapısını gösterir.

Sunumda tablo yapısını göstermek için şu komut yararlı olabilir:

```
\d uyelik
```

Bu komut uyelik tablosundaki kolonları, constraint'leri ve index'leri gösterir.

Bölüm 33

Neden Raw SQL Kullanıldı?

Python web projelerinde veritabanı için iki yaygın yaklaşım vardır:

1. Raw SQL yazmak
2. ORM kullanmak

ORM, Python class'ları üzerinden veritabanı işlemi yapmayı sağlar. Ancak bu proje bir veritabanı dersi projesi olduğu için raw SQL daha uygundur.

Raw SQL kullanmanın avantajı:

- Gerçek SQL sorguları açıkça görünür.
- JOIN, GROUP BY, CHECK, TRIGGER, INDEX gibi konular doğrudan anlatılır.
- Sunumda veritabanı bilgisi daha net gösterilir.
- `schema.sql` ve uygulamadaki sorgular ders konularıyla uyumludur.

Bu yüzden `db.py` basit bağlantı yardımcıları sağlar, ama sorgular `app.py` içinde açık SQL olarak yazılır.

Örnek:

```
rows = fetch_all(
    """
    SELECT p.plan_adi, COUNT(*) AS adet
      FROM uyelik u
     JOIN plan p ON p.plan_id = u.plan_id
    WHERE u.durum = 'AKTIF'
    GROUP BY p.plan_adi
    """
)
```

Bu kodda SQL mantığı açıkça görülebilir.

Bölüm 34

Projede Bilerek Basit Tutulan Noktalar

Bu proje ders ve sunum odaklı olduğu için bazı konular bilinçli olarak basit tutulmuştur.

Konu	Mevcut durum	Gerçek ürün olsaydı
Kullanıcı rolleri	Sadece yönetici login var	PT, diyetisyen ve resepsiyon için ayrı roller eklenebilirdi
CSRF koruması	Eklenmedi	Flask-WTF veya benzeri yapı kullanılabilirdi
Şifre değiştirme	Yok	Yönetici profil ekranı eklenebilirdi
Production sunucu	Flask development server	Gunicorn/uWSGI ve reverse proxy kullanılabilirdi
Migration sistemi	schema.sql ile kurulum	Alembic veya migration dosyaları kullanılabilirdi

Bu eksikler projenin amacına ters değildir. Çünkü bu projenin ana hedefi ilişkisel veritabanı tasarımı, SQL sorguları, iş kuralları ve çalışan bir yönetim paneli göstermektir.

Bölüm 35

Sonuç

Bu proje, yalnızca arayüzü olan basit bir web uygulaması değildir. Aynı zamanda ilişkisel veritabanı tasarımını, SQL sorgularını, veri bütünlüğü kurallarını ve backend/frontend etkileşimini birlikte gösteren bütünlüklü bir örnektir.

Flask tarafı kullanıcı isteklerini yönetir. PostgreSQL tarafı veriyi ve iş kurallarını korur. HTML/CSS/Jinja tarafı kullanıcıya düzenli bir arayüz sunar. Chart.js ise verileri grafiklerle anlaşılır hale getirir.

Bu yüzden proje, veritabanı dersi için hem teknik hem görsel olarak güçlü bir sunum malzemesi oluşturur.

Kapanış Notu

Bu doküman, projenin kaynak kodu ve veritabanı tasarımı ile birlikte okunmak üzere hazırlanmıştır. Kod örnekleri, tablolar ve açıklamalar aktif proje yapısına göre düzenlenmiştir.